

Actuator:

Provides production-ready endpoints to monitor and manage the Spring boot application.

Project setup:

Project
☐ Gradle - Groovy ☐ Gradle - Kotlin ☒ **Maven**

Language
☒ **Java** ☐ Kotlin ☐ Groovy

Spring Boot
☐ 4.0.0 (SNAPSHOT) ☐ 4.0.0 (M3) ☐ 3.5.7 (SNAPSHOT) ☒ **3.5.6**
☐ 3.4.11 (SNAPSHOT) ☐ 3.4.10

Project Metadata

Group

Artifact

Name

Description

Package name

Packaging ☒ **Jar** ☐ War

Java ☐ 25 ☐ 21 ☒ **17**

Dependencies ADD DEPENDENCIES... ⌘ + B

Spring Web **WEB**
Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

Spring Boot Actuator **OPS**
Supports built in (or custom) endpoints that let you monitor and manage your application - such as application health, metrics, sessions, etc.

application.properties

```
application.properties ×
1  spring.application.name=order-service
2
3  # by-default path is /actuator
4  management.endpoints.web.base-path=/manage
5
6  # Expose all actuator endpoints, by-default '/actuator/health' & '/actuator/info' endpoint is exposed
7  # '*' expose all the endpoints
8  # use comma separated like: health, info, metrics, loggers etc. to expose selected endpoints
9  management.endpoints.web.exposure.include=*
10
11
```

Let's start the server and hit the Actuator endpoints:

```
6  @SpringBootApplication
7  > public class ActuatorApplication {
8
9  >     public static void main(String[] args) { SpringApplication.run(ActuatorApplication.class, args); }
12 }
```

Run ConfigServerApplication x ActuatorApplication x

Spring Boot (v3.5.6)

```
2025-10-02T13:59:09.597+05:30 INFO 55995 --- [order-service] [main] com.concepts.ActuatorApplication : Starting ActuatorApplication using Java
2025-10-02T13:59:09.600+05:30 INFO 55995 --- [order-service] [main] com.concepts.ActuatorApplication : No active profile set, falling back to
2025-10-02T13:59:10.014+05:30 INFO 55995 --- [order-service] [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http
2025-10-02T13:59:10.019+05:30 INFO 55995 --- [order-service] [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2025-10-02T13:59:10.019+05:30 INFO 55995 --- [order-service] [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat
2025-10-02T13:59:10.035+05:30 INFO 55995 --- [order-service] [main] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplica
2025-10-02T13:59:10.036+05:30 INFO 55995 --- [order-service] [main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initializat
2025-10-02T13:59:10.246+05:30 INFO 55995 --- [order-service] [main] o.s.b.a.e.web.EndpointLinksResolver : Exposing 13 endpoints beneath base path
2025-10-02T13:59:10.269+05:30 INFO 55995 --- [order-service] [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with
2025-10-02T13:59:10.276+05:30 INFO 55995 --- [order-service] [main] com.concepts.ActuatorApplication : Started ActuatorApplication in 0.84 sec
```

Health API : '/health'

localhost:8080/manage/health

Pretty print ☒

```
{
  "status": "UP"
}
```

Metrics API : '/metrics'

/metrics' gives list of all metrics endpoint

← → ↻ ⓘ localhost:8080/manage/metrics

Pretty print ☒

```
{
  "names": [
    "application.ready.time",
    "application.started.time",
    "disk.free",
    "disk.total",
    "executor.active",
    "executor.completed",
    "executor.pool.core",
    "executor.pool.max",
    "executor.pool.size",
    "executor.queue.remaining",
    "executor.queued",
    "http.server.requests",
    "http.server.requests.active",
    "jvm.buffer.count",
    "jvm.buffer.memory.used",
    "jvm.buffer.total.capacity",
    "jvm.classes.loaded",
    "jvm.classes.unloaded",
    "jvm.compilation.time",
    "jvm.gc.concurrent.phase.time",
    "jvm.gc.live.data.size",
    "jvm.gc.max.data.size",
    "jvm.gc.memory.allocated",
    "jvm.gc.memory.promoted",
    "jvm.gc.overhead",
    "jvm.gc.pause",
    "jvm.info",
    "jvm.memory.committed",
    "jvm.memory.max",
    "jvm.memory.usage.after.gc",
    "jvm.memory.used",
    "jvm.threads.daemon",
    "jvm.threads.live",
    "jvm.threads.peak",
    "jvm.threads.started",
    "jvm.threads.states",
    "logback.events",
    "process.cpu.time",
    "process.cpu.usage",
    "process.files.max",
    "process.files.open",
    "process.start.time",
    "process.uptime",
    "system.cpu.count",
    "system.cpu.usage",
    "system.load.average.1m",
    "tomcat.sessions.active.current",
    "tomcat.sessions.active.max",
    "tomcat.sessions.alive.max",
    "tomcat.sessions.created",
    "tomcat.sessions.expired",
    "tomcat.sessions.rejected"
  ]
}
```



```
localhost:8080/manage/metrics/executor.pool.core

Pretty print ☒

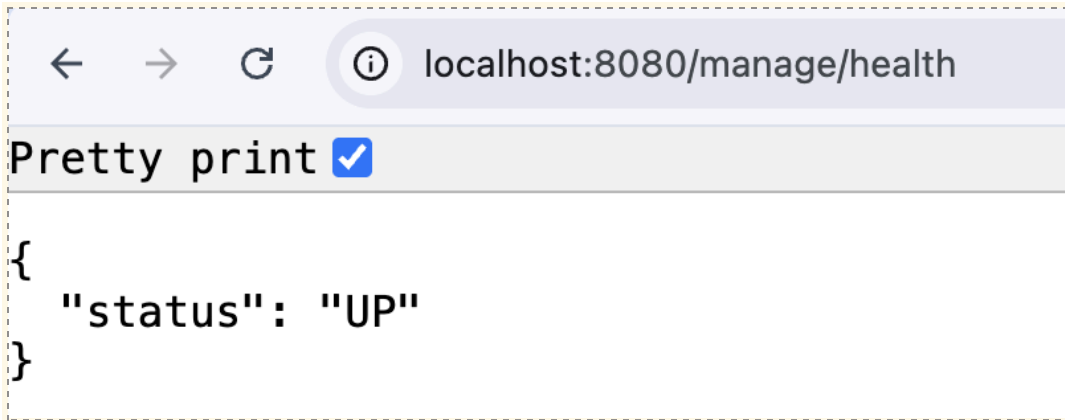
{
  "name": "executor.pool.core",
  "description": "The core number of threads for the pool",
  "baseUnit": "threads",
  "measurements": [
    {
      "statistic": "VALUE",
      "value": 8
    }
  ],
  "availableTags": [
    {
      "tag": "name",
      "values": [
        "applicationTaskExecutor"
      ]
    }
  ]
}
```

Hitting specific metrics endpoint

Lets first see some useful in-built Actuator endpoints:

GET: /health

Provides health status of our application: UP, DOWN, OUT_OF_SERVICE, UNKNOWN



/health endpoint can be extended to add additional checks like DB health, Cache status etc. beyond just application server status

Adding DB health check:

```
@Component
public class DatabaseHealthIndicator implements HealthIndicator {
    @Override
    public Health health() {
        boolean isDBUp = checkDBConnection();
        return isDBUp ? Health.up().withDetail("DB", "Available").build()
            : Health.down().withDetail("DB", "Not-Available").build();
    }

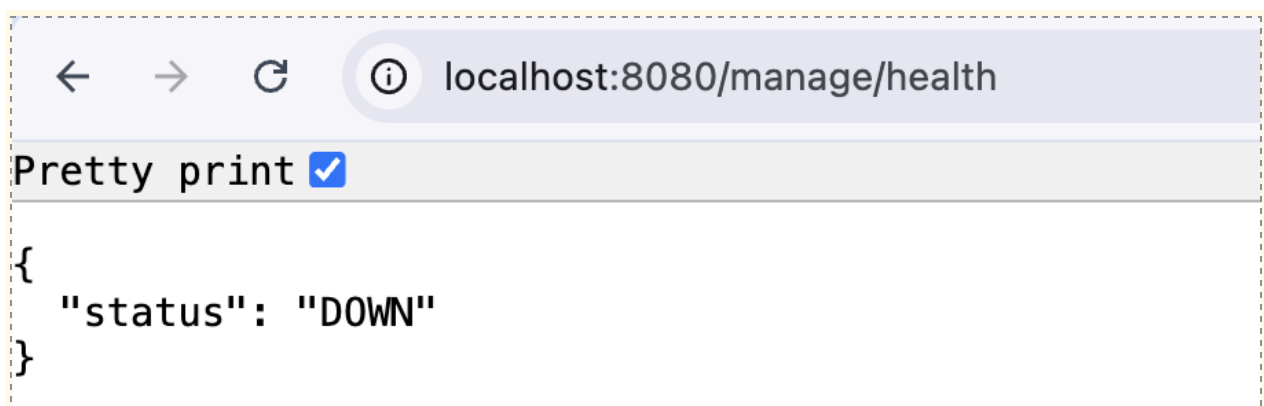
    private boolean checkDBConnection() {
        //check DB is up or not
        return true;
    }
}
```

Adding Cache health check:

```
@Component
public class CacheHealthIndicator implements HealthIndicator {
    @Override
    public Health health() {
        boolean isCacheUp = checkCacheStatus();
        return isCacheUp ? Health.up().withDetail("Cache", "Available").build()
            : Health.down().withDetail("Cache", "Not-Available").build();
    }

    private boolean checkCacheStatus() {
        //check cache status
        return false;
    }
}
```

By-default, its shows only the overall status of the application



But if we want more details, we need to add:

application.properties ×

```
1 spring.application.name=order-service
2
3 # by-default path is /actuator
4 management.endpoints.web.base-path=/manage
5
6 # Expose all actuator endpoints, by-default '/actuator/health' & '/actuator/info' endpoint is exposed
7 # '*' expose all the endpoints
8 # use comma separated like: health, info, metrics, loggers etc. to expose selected endpoints
9 management.endpoints.web.exposure.include=*
10
11 #to view the details in health endpoint
12 management.endpoint.health.show-details=always
13
```



localhost:8080/manage/health

Pretty print ☒

```
{
  "status": "DOWN",
  "components": {
    "cache": {
      "status": "DOWN",
      "details": {
        "Cache": "Not-Available"
      }
    },
    "database": {
      "status": "UP",
      "details": {
        "DB": "Available"
      }
    }
  }
}
```


GET: /metrics
and
/metrics/{metric name}

There are so many metrics, just showing some important ones:

	Metric name	What it represents	Example
JVM Memory Metrics			
	jvm.memory.usage	Memory currently in use by JVM	{ "statistic": "VALUE", "value": 99478616 }

	jvm.memory.max	Max memory JVM can use	{ "statistic": "VALUE", "value": 10989076477 }
Garbage Collection Metrics			

	jvm.gc. pause	Time spent in GC. COUNT: total no. of GC events occurred. TOTAL_TIME: total time spent in GC (usually seconds) MAX: longest single GC pause observed	[{ COUNT: total no. of GC events occurred. "statistic": "COUNT", "value": 12 }, { MAX: longest single GC pause observed "statistic": "TOTAL_TIME", "value": 2.305 }, { "statistic": "MAX", "value": 0.9 }]
Threads			

	jvm.threads.live	Number of live threads	{ "statistic": "VALUE", "value": 22 }
	jvm.threads.peak	Peak live thread count, since JVM started	{ "statistic": "VALUE", "value": 50 }
System Metrics			
	system.cpu.usage	CPU used by JVM (range 0.0 - 1.0)	Value: 0.10 -> 10%
HTTP Server / Requests			

	http.server.requests	COUNT: Total no of HTTP requests received TOTAL_TIME: total time spent handling all requests (usually in seconds) MAX: longest time taken to handle a single request	<pre> "measurements": [{ "statistic": "COUNT", "value": 152 }, { "statistic": "TOTAL_TIME", "value": 23.45 }, { "statistic": "MAX", "value": 0.89 }], "availableTags": [{ "tag": "method", "values": ["GET", "POST"]} }, </pre>
--	----------------------	--	---

			<pre>{ "tag": "status", "values": ["200", "404", "500"] }] }</pre>
Database /JDBC Metric			
	jdbc.co nnectio ns.activ e	Connections currently in use	VALUE: 3
	jdbc.co nnectio ns.idle	Idle connections in pool	VALUE: 7
	jdbc.co nnectio ns.max	Max connections allowed	VALUE: 10

GET: /threaddump

- Helps to diagnose deadlock or thread leaks.

- Which threads are active, blocked or waiting.
- Stack trace for each thread (what code it is executing)
- Thread name, id, priority etc.

```
[
  {
    "threadName": "main",
    "threadId": 1,
    "blockedTime": -1,
    "blockedCount": 0,
    "waitedTime": -1,
    "waitedCount": 0,
    "threadState": "RUNNABLE",
    "stackTrace": [
      "com.concepts.MyService.methodName(MyService.java:142)",
      "com.concepts.ActuatorApp.main(ActuatorApp.java:10)"
    ]
  },
  {
    "threadName": "thread2",
    "threadId": 2,
    "blockedTime": -2,
    "blockedCount": 0,
    "waitedTime": -1,
    "waitedCount": 0,
    "threadState": "WAITING",
    "stackTrace": [
      "com.concepts.ClassName.methodName(ClassName.java:12)",
      "com.concepts.ActuatorApp.main(ActuatorApp.java:10)"
    ]
  }
]
```

What all other API's available:

Official Documentation:

<https://docs.spring.io/spring-boot/reference/actuator/endpoints.html#actuator.endpoints>

/heapdump	GET	Downloads JVM heap as .hprof file
/mappings	GET	Lists all Spring MVC request mappings
/beans	GET	Displays a complete list of all the Spring beans in your application
/configprops	GET	Lists all @ConfigurationProperties beans
/loggers	GET	Lists all loggers and their current levels
/shutdown	POST	Lets the application be gracefully shutdown
/env	GET	Shows environment properties
/actuator/env/{property}	GET	Shows a specific environment property

By default, access to all endpoints except for **"/shutdown"** and **"/heapdump"** is unrestricted

Both these endpoints are very critical:

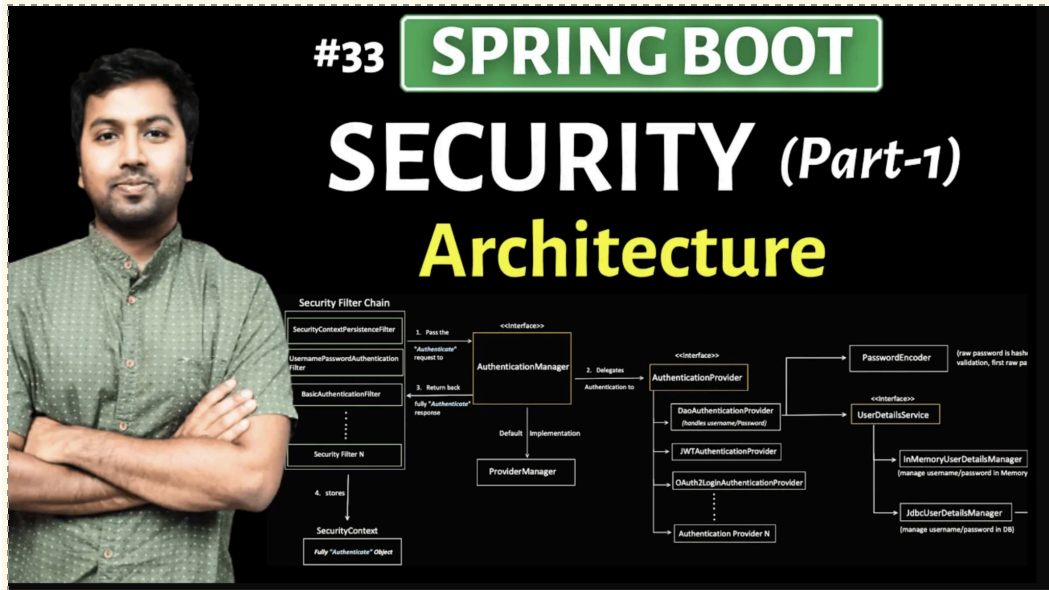
- **/shutdown** can stop your application
- **/heapdump** can expose sensitive information about your application like tokens, password etc.

So, by default they are restricted and specifically want us to unrestricted it (assuming, we have accepted the risk involved)

```
application.properties ×
2
3 # by-default path is /actuator
4 management.endpoints.web.base-path=/manage
5
6 # Expose all actuator endpoints, by-default '/actuator/health' & '/actuator/info' endpoint is exposed
7 # '*' expose all the endpoints
8 # use comma separated like: health, info, metrics, loggers etc. to expose selected endpoints
9 management.endpoints.web.exposure.include=*
10
11 #to view the details in health endpoint
12 management.endpoint.health.show-details=always
13
14 # Make it require authentication
15 management.endpoint.health.access=unrestricted
16
17
```

Security

We have already covered Spring Security in depth (total 9 parts), so kindly check it out, if there is any doubt with that:



pom.xml

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

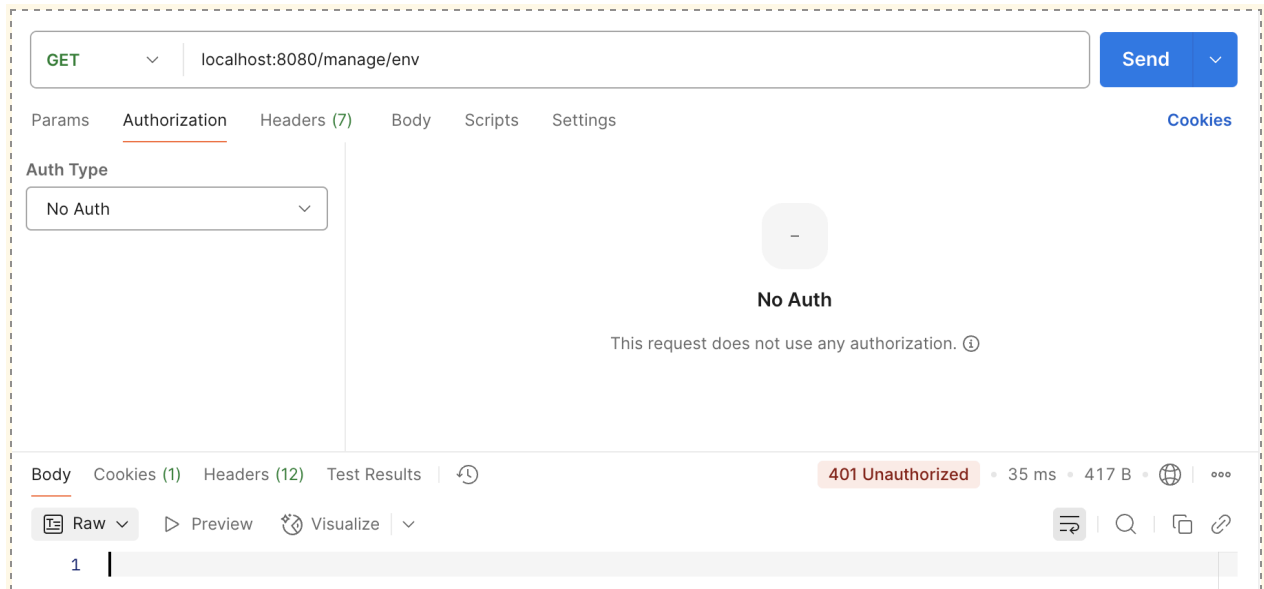
    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception{

        http.authorizeHttpRequests(auth -> auth
            .requestMatchers(...patterns: "/manage/health", "/manage/info").permitAll() // public
            .anyRequest().authenticated() // everything else requires login
        )
        .httpBasic(Customizer.withDefaults()); // basic auth for simplicity
        return http.build();
    }
}
```

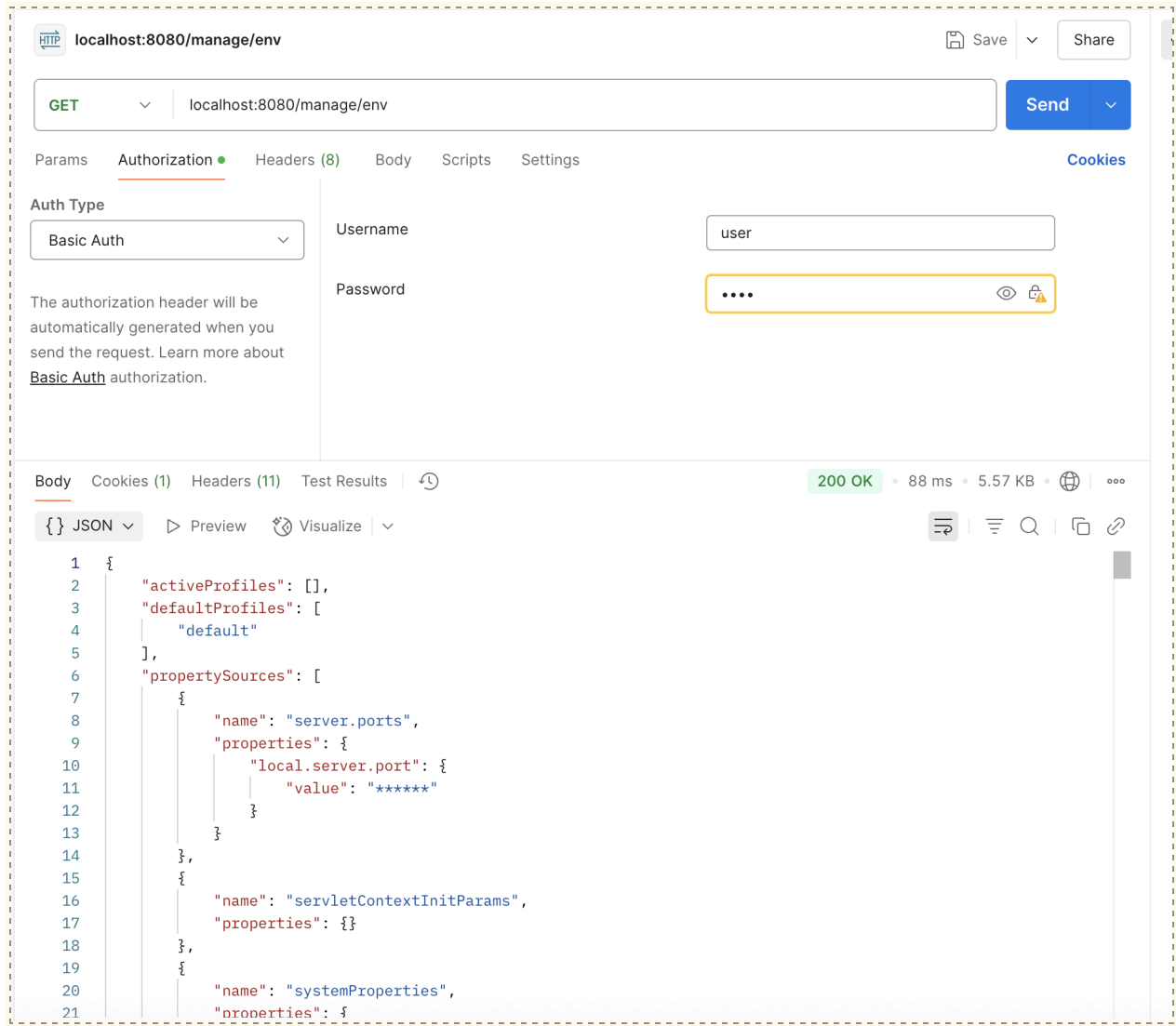
application.properties

```
spring.security.user.name=user  
spring.security.user.password=pass  
spring.security.user.roles=ADMIN
```

Trying to access /env without providing the authentication, request denied:



Able to access, after successful Authentication:



Custom Actuator Endpoint:

Class annotated with `@Endpoint(id = "custom endpoint name")`

@Component

@Endpoint(id = "my-custom-stats")

public class MyCustomStatsEndpoint { ... }

Type of operation, our custom endpoint going to support:

@ReadOperation

(equivalent to HTTP GET)

@WriteOperation

(equivalent to HTTP POST)

@DeleteOperation

(equivalent to HTTP DELETE)

Component

@Endpoint(id = "my-custom-stats")

public class MyCustomStatsEndpoint {

@ReadOperation

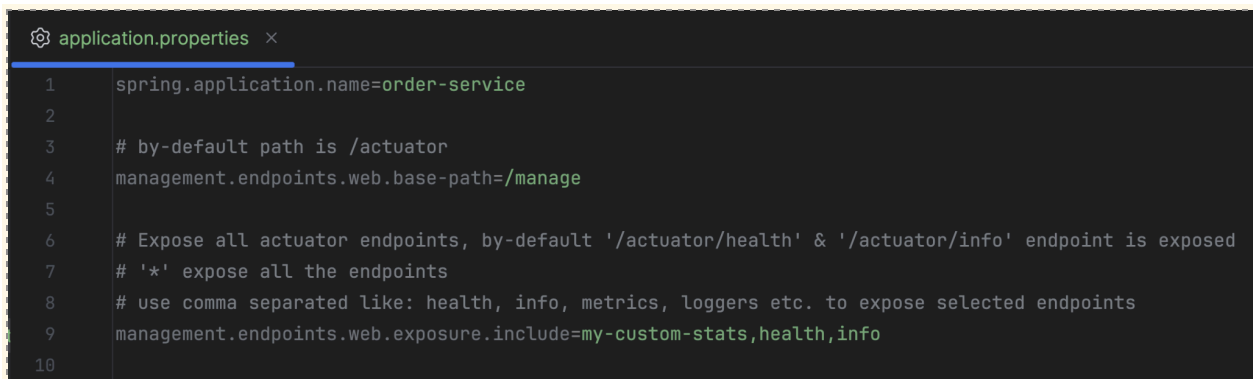
```
public String readAll() {  
    return "Hello, Spring Boot!";  
}
```

@ReadOperation

```
public String read(@Selector String name,  
@Selector String message) {  
    return "Hello: " + name + " msg for you is: " +  
message;  
}  
}
```

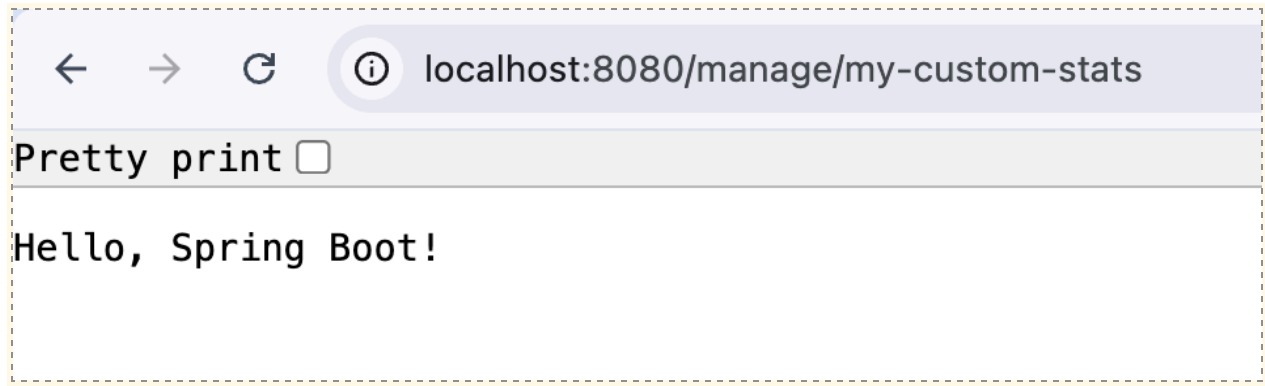
Return type can be any serializable object like Map, List, String, POJO, primitive etc.

***/my-custom-stats/{name}/{message}
selector follows sequence***

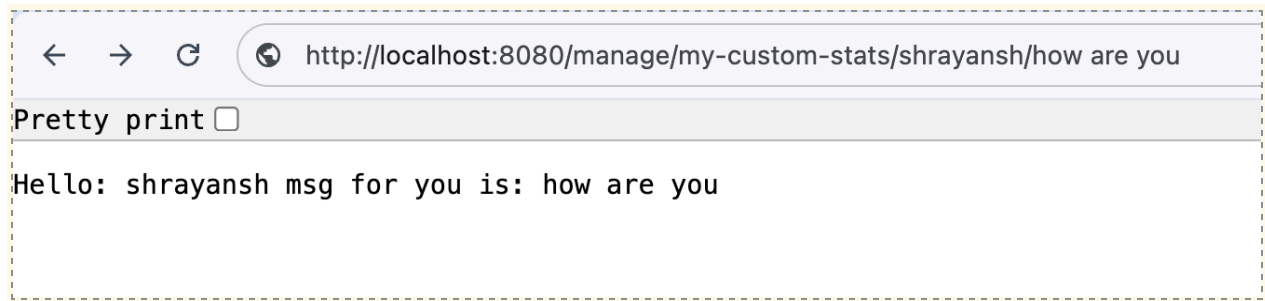
A screenshot of a code editor showing the 'application.properties' file. The file contains configuration for a Spring application, including the application name, management endpoints, and exposure settings. The text is as follows:

```
1 spring.application.name=order-service  
2  
3 # by-default path is /actuator  
4 management.endpoints.web.base-path=/manage  
5  
6 # Expose all actuator endpoints, by-default '/actuator/health' & '/actuator/info' endpoint is exposed  
7 # '*' expose all the endpoints  
8 # use comma separated like: health, info, metrics, loggers etc. to expose selected endpoints  
9 management.endpoints.web.exposure.include=my-custom-stats,health,info  
10
```

GET:



GET:



For POST and DELETE operation, it requires authentication

@Component

@Endpoint(id = "my-custom-stats")

public class MyCustomStatsEndpoint {

@WriteOperation


```
public String refresh() {  
    // simulate say cache refresh  
    return "refreshed";  
}
```

@DeleteOperation

```
public String remove(@Selector String key) {  
    return "reset done for key: " + key;  
}  
}
```

application.properties

```
application.properties ×  
1  spring.application.name=order-service  
2  
3  # by-default path is /actuator  
4  management.endpoints.web.base-path=/manage  
5  
6  # Expose all actuator endpoints, by-default '/actuator/health' & '/actuator/info' endpoint is exposed  
7  # '*' expose all the endpoints  
8  # use comma separated like: health, info, metrics, loggers etc. to expose selected endpoints  
9  management.endpoints.web.exposure.include=my-custom-stats,health,info  
10  
11  spring.security.user.name=user  
12  spring.security.user.password=pass  
13  spring.security.user.roles=ADMIN  
14
```

POST

localhost:8080/manage/my-custom-stats

Send

Params

Authorization

Headers (9)

Body

Scripts

Settings

Cookies

Auth Type

Basic Auth

The authorization header will be automatically generated when you send the request. Learn more about [Basic Auth](#) authorization.

Username

user

Password

....

Body

Cookies (1)

Headers (11)

Test Results

200 OK

93 ms

362 B

{ } JSON

Preview

Visualize

1 refreshed

DELETE

localhost:8080/manage/my-custom-stats/myKey

Send

Params

Authorization

Headers (8)

Body

Scripts

Settings

Cookies

Auth Type

Basic Auth

The authorization header will be automatically generated when you send the request. Learn more about [Basic Auth](#) authorization.

Username

user

Password

....

Body

Cookies (1)

Headers (11)

Test Results

200 OK

101 ms

379 B

{ } JSON

Preview

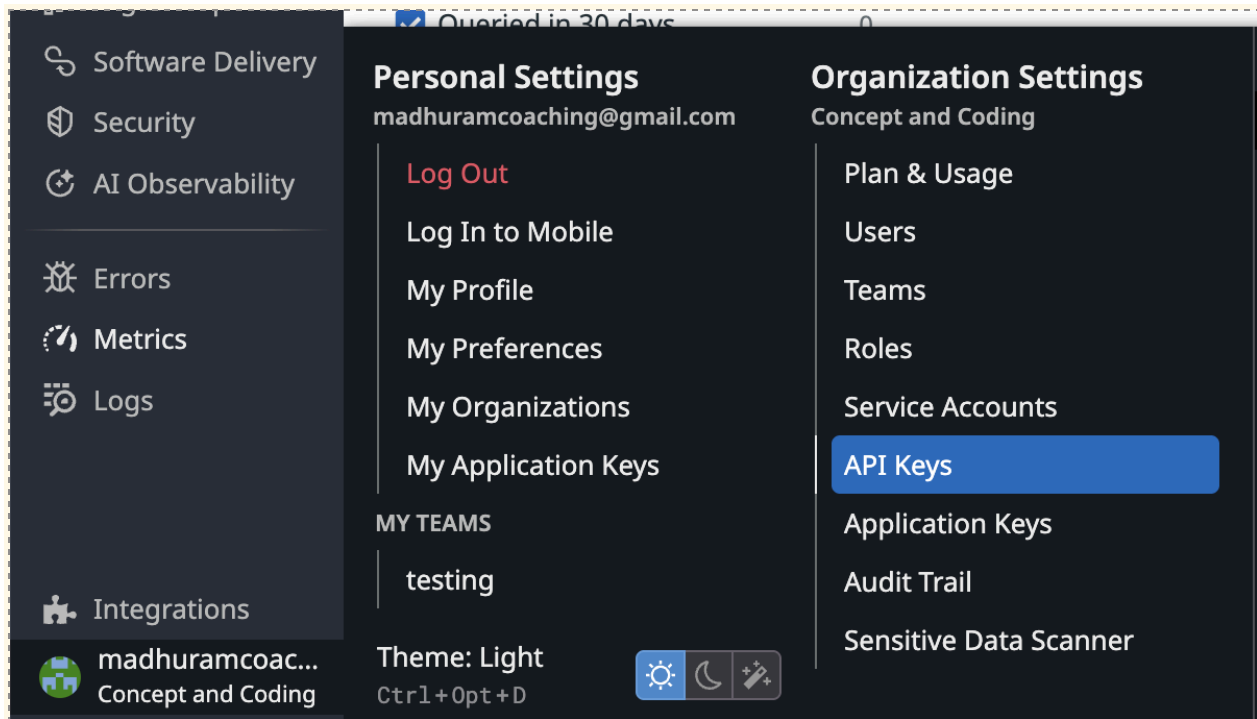
Visualize

1 reset done for key: myKey

Pushing the metrics to Datadog (Monitoring platform)

we can also push to different other platforms like: Prometheus, CloudWatch etc..

Datadog (we can test with Free trial)



API Keys

+ New Key

application.properties

```
management.datadog.metrics.export.apiKey=e59af51cb6ae3a24129da7230e5dd9f7  
management.datadog.metrics.export.enabled=true  
management.datadog.metrics.export.step=5s
```

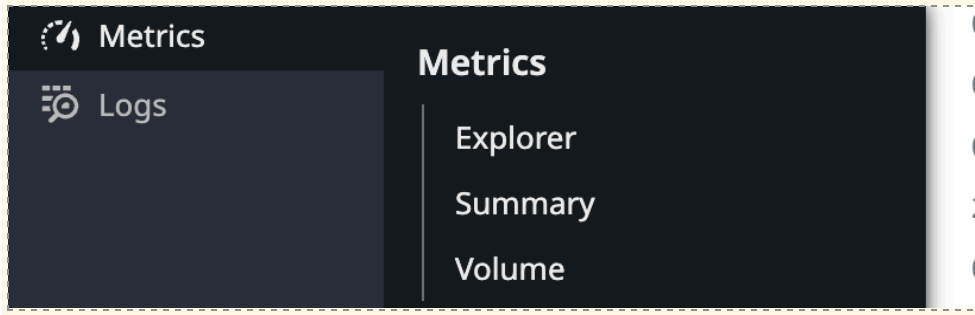
***When its true then only it try to push the metrics
Every 5s it will push the metrics to datadog***

POm.xml

```
<dependency>  
  <groupId>io.micrometer</groupId>  
  <artifactId>micrometer-registry-datadog</artifactId>  
</dependency>
```

**All the metrics of Spring Boot Actuator are
automatically pushed to Datadog.**

**choose the metric we want to monitor
for example: [http.server.requests.count](#)**



choose the metric we want to monitor
for example: *http.server.requests.count*

